Aplicación de métodos estadísticos en la validación automatizada y auditoría de órdenes de trabajo en ENEL

Dayana Alejandra Rincón Pardo

Facultad de estadística, Universidad Santo Tomás

Práctica profesional

Dr. Wilmer Darío Pineda Ríos

22 de noviembre del 2025

CONTENIDO

1. OBJETIVOS 5
a. GENERAL 5
b. ESPECÍFICOS 5
2. MARCO TEÓRICO 6
a. Calidad de datos en sistemas empresariales 6
b. Pipelines estadísticos para validación de datos 10
c. Validación automatizada mediante pruebas estadísticas 11
d. Medidas de concordancia para datos categóricos 16
3. METODOLOGÍA 16
4. RESULTADOS PRELIMINARES 36
5. Resultados del pipeline de validación 42
6. CONCLUSIONES 44
7. REFERENCIAS 49

1. PLANTEAMIENTO FORMAL DEL PROBLEMA

La confiabilidad de los datos constituye un requisito fundamental para asegurar eficiencia, transparencia y control en las organizaciones. En el caso de ENEL, la información vinculada a los diferentes procesos se origina en fuentes heterogéneas como SCM la cuál es una herramienta fundamental en donde se realiza la validación de cada ticket (Un ticket es una solicitud de pago a contratistas que revisan un determinado problema, hacer una reparación o mantenimiento a un punto de un circuito eléctrico ubicado en una zona) , en Forcebeat se revisa el registro fotográfico de la incidencia con su respectivo trabajo realizado por parte del contrato, baremos son actividades que se realizan, que actualmente se procesan de manera manual. Este procedimiento incrementa los tiempos de gestión, aumenta la probabilidad de errores de conciliación y limita la trazabilidad de los procesos de facturación.

Desde una perspectiva estadística, esta situación representa un problema de calidad de datos, evidenciado en la existencia de valores faltantes, registros duplicados, inconsistencias entre fuentes y presencia de valores atípicos. Estos defectos afectan la validez de los análisis posteriores, reducen la capacidad de auditoría y restringen la toma de decisiones tanto operativas como estratégicas.

La literatura académica ofrece lineamientos para abordar esta problemática. Oyoo y Berleant (2021) resaltan que, en organizaciones de energía y utilities, resulta necesario garantizar dimensiones de calidad como completitud, consistencia y unicidad mediante mecanismos automáticos de validación. De forma complementaria, Mattila (2023) plantea un modelo de seis etapas para integrar la validación estadística en pipelines de datos, lo cual permite detectar inconsistencias de manera temprana y documentar los resultados de forma estructurada. En la misma línea, Redyuk, Kaoudi y Schelter (2021) proponen un enfoque basado en la detección

automática de anomalías, sustentado en estadísticas descriptivas y aprendizaje adaptativo, que reduce la dependencia de reglas manuales en entornos de ingesta dinámica.

Con base en estos aportes, el presente proyecto propone diseñar e implementar un pipeline en Python que consolide las distintas fuentes de información, ejecute validaciones sobre dimensiones críticas de calidad de datos, aplique técnicas de detección de valores atípicos y genere reportes de auditoría con trazabilidad verificable.

1. OBJETIVOS

a. GENERAL

Evaluar la calidad de los datos generados en los procesos operativos de ENEL a partir de criterios estadísticos de completitud, consistencia y trazabilidad, con el fin de fortalecer la confiabilidad de la información utilizada en la gestión y toma de decisiones.

b. ESPECÍFICOS

- Consolidar las distintas fuentes de información operativa de ENEL (SCM, Forcebeat, Baremos) en un flujo de datos unificado y estructurado que permita su análisis integral.
- Establecer criterios y métricas estadísticas para evaluar dimensiones críticas de calidad de datos, tales como completitud, consistencia y unicidad.
- Aplicar técnicas estadísticas de detección de valores atípicos y anomalías con el propósito de identificar posibles irregularidades en los registros provenientes de las diferentes fuentes.
- Desarrollar un sistema de reporte que documente los resultados de las validaciones y permita la trazabilidad de los procesos del control de calidad.

2. MARCO TEÓRICO

a. Calidad de datos en sistemas empresariales

La calidad de datos se define como el grado en que un conjunto de información es adecuado para su propósito analítico u operativo (Wang & Strong, 1996). En sistemas operativos de empresas de energía —incluyendo SCM y Forcebeat— la calidad de datos es un elemento crítico para asegurar facturación precisa, auditoría verificable y toma de decisiones basada en evidencia.

Dimensiones de calidad según ISO 8000-2

1. **Completitud (Completeness)**

   Proporción de valores observados frente al total esperado.

   Sea X un campo con n registros:

$$C(X) = \frac{n - n_{\text{missing}}}{n}$$

2. **Unicidad (Uniqueness)**

   Evalúa la ausencia de registros duplicados en atributos que deben identificar entidades únicas:

$$U(X) = \frac{n_{\text{unique}}}{n}$$

3. **Consistencia (Consistency)**

   Grado de acuerdo entre fuentes que describen la misma unidad de análisis.

   Kappa mide el grado de concordancia de las evaluaciones nominales u ordinales realizadas por múltiples evaluadores cuando se evalúan las mismas muestras.

Para variables categóricas, se emplea el coeficiente **Kappa de Cohen** (Cohen, 1960):

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

donde:

- p_o: acuerdo observado
- p_e: acuerdo esperado por azar

Ejemplo:

Dos médicos diferentes examinan a 45 pacientes para determinar si tienen una enfermedad específica. ¿Con qué frecuencia coincidirá el diagnóstico de los médicos con respecto a la enfermedad (positivo o negativo)?

Tabla 1

Clasificación del valor Kappa (Landis & Koch, 1977):

VALOR DE KAPPA	INTERPRETACIÓN
0 - 0.20	Acuerdo leve
0.21 - 0.40	Aceptable
0.41 - 0.60	Moderado
0.61 - 0.80	Sustancial
0.81 - 1.00	Casi perfecto

Nota. Tomado de The Measurement of Observer Agreement for Categorical Data

Kappa = 1 → Acuerdo perfecto

Kappa = 0 → Acuerdo igual al azar

Kappa < 0 → Peor que el azar

4. **Exactitud (Accuracy)**

   Cercanía entre el valor registrado y la realidad empírica.

5. **Actualidad (Timeliness)**

   Oportunidad del dato respecto a la dinámica del proceso.

6. **Validez (Validity)**

   Cumplimiento de reglas de negocio, rangos y formatos establecidos.

Mecanismos de datos faltantes (Little & Rubin, 2019)

1. **MCAR (Missing Completely at Random)**

   El dato falta completamente al azar, la ausencia no depende en este caso de ninguna variable observada o del valor que falta.

   Ejemplo:

   Una tableta de campo se apaga por batería baja y al azar deja de registrar la fecha de llegada del técnico para algunos tickets.

   No depende del tipo de incidencia, del contratista, ni de la zona.

   Es completamente accidental.

2. **MAR (Missing at Random)**

   El dato falta de manera condicionada a variables observadas, la ausencia si depende de algo registrado, pero no del valor faltante en sí.

Ejemplo:

Supongamos que los tickets atendidos en zonas rurales tienen peor conectividad. Por esta razón el campo “Foto del trabajo ejecutado” falta más en zonas rurales. Aquí la ausencia depende exclusivamente de la zona y no depende del contenido faltante.

3. **MNAR (Missing Not at Random)**

   El dato falta porque el valor faltante en sí mismo influye en su ausencia. Esta depende de algo no observado o del propio valor faltante.

   Ejemplo:

   Si un contratista omite el registro de una foto cuando el trabajo quedó mal hecho: La ausencia depende de la calidad del trabajo que no está registrada, justo los casos con un resultado “negativo” tienen más riesgo de faltar.

4. **MNAR por diseño**

   El dato faltante es intencional, producto generalmente de reglas de negocio. No existe porque no aplica.

   Ejemplo:

   Un campo “código de cabina” solo aplica para instalaciones con transformadores, para postes simples o acometidas el campo siempre estará vacío porque no tiene sentido registrarlo.

Índice agregado de calidad (Batini et al., 2021)

Este índice (DQ) mide la calidad global de algo por ejemplo (datos, procesos, servicios, operaciones etc.)

$$DQ = w_1C + w_2U + w_3Cons + w_4A + w_5T + w_6V$$

Los pesos $w_i$se ajustan según criticidad operativa.

Tabla 2

Dimensiones del índice DQ (Batini et al., 2021):

Simbolo	Dimensión	Significado
C	Completitud	Completitud de los datos o informacion.
U	Unicidad	Ausencia de duplicados.
Cons	Consistencia	Coherencia entre reglas lógicas, formatos y valores.

Nota. Tomado de *Batini, C., & Scannapieco, M. (2021). Data and Information Quality. Springer.*

b. Pipelines estadísticos para validación de datos

Un pipeline corresponde a una secuencia reproducible de transformaciones que convierten datos operativos en información validada y lista para análisis (Mattila, 2023).

Arquitectura ETL con validaciones integradas

1. **Extract**

   Integración de datos desde SCM, Forcebeat u otras fuentes, considerando validación de formatos y lectura robusta.

2. **Transform**

   - Estandarización de fechas, codificaciones y tipos
   - Eliminación de duplicados exactos

     Creación de variables derivadas:

   - delta temporal, indicadores de control, flags de coherencia territorial

3. **Validate**
    - Pruebas de hipótesis sobre completitud
    - Outliers univariados y multivariados
    - Validación de reglas de dominio
    - Consistencia SCM–Forcebeat
4. **Load**

    Almacenamiento del dataset validado con metadata de calidad (logs, flags, versiones).

Principios de diseño

- **Modularidad:** Cada operación se implementa como función independiente.
- **Idempotencia:** La ejecución repetida sobre el mismo input produce el mismo output.
- **Trazabilidad:** Se registran parámetros, resultados, fechas y usuario.
- **Reproducibilidad:** Versionamiento de datos (DVC), código (Git) y semillas aleatorias.
- **Control de fuga de información:**

  Crucial cuando intervienen modelos ML como Isolation Forest. Las transformaciones se calibran solo con datos de referencia (train).

c. Validación automatizada mediante pruebas estadísticas

La validación basada en estadística reemplaza reglas manuales por criterios objetivos con propiedades inferenciales.

Pruebas de hipótesis sobre completitud

Para un campo crítico X:

$$H_0: p_{\text{missing}} \leq p_0$$

$p_{missing}$: proporción real de datos faltantes en ese campo.

p_0 : máximo porcentaje tolerable de datos faltantes.

Bajo la hipótesis nula, el campo es suficientemente completo. La proporción de faltantes es igual o menor al umbral permitido.

La prueba binomial estima:

$$P(N_{\text{miss}} \geq k \mid n, p_0) = \sum_{i=k}^{n} \binom{n}{i} p_0^i (1 - p_0)^{n-i}$$

Si p-valor $< \alpha$: se rechaza H_0 → completitud insuficiente.

Corrección por comparaciones múltiples

- Bonferroni: $\alpha_{\text{adj}} = \alpha / k$
- Benjamini–Hochberg: controla FDR con mayor potencia.

Outliers univariados

Son valores demasiado extremos en una sola variable (por ejemplo, ingresos, edad, consumo, visitas, etc.).

1. **Método IQR (Tukey, 1977)**

$$\text{LB} = Q_1 - k \cdot IQR, \text{UB} = Q_3 + k \cdot IQR$$

Este método usa la **distribución de los datos**, en particular los *cuartiles*:

Q_1 : primer cuartil (25%)

Q_3 : tercer cuartil (75%)

IQR: $Q_3 - Q_1$: $rango\ intercuartílico$

El criterio define límites:

LB = $Q_1 - K * IQR$

UB = $Q_3 + K * IQR$

- LB (Lower Bound) → límite inferior
- UB (Upper Bound) → límite superior
- k suele ser:
    - 1.5 para detectar outliers moderados
    - 3 para detectar outliers extremos

Cualquier valor fuera de [LB, UB] se considera un outlier.

2. **Z-score**

$$z_i = \frac{x_i - \mu}{\sigma}$$

- x_i: Valor individual
- μ: media
- σ: desvisación estándar

El *z-score* indica cuántas desviaciones estándar se aleja un valor del promedio.

Se considera outlier si $| z_i | > 3$.

Outliers multivariados

En análisis multivariado, un outlier es una observación que se desvía de forma significativa del patrón general de los datos considerando simultáneamente todas las variables. A diferencia del análisis univariado, en el que un valor extremo se identifica respecto a una sola dimensión, en el caso multivariado una observación puede no ser extrema en ninguna variable individual, pero sí en su combinación conjunta.

Uno de los métodos modernos más utilizados para detectar outliers multivariados en grandes volúmenes de datos es Isolation Forest.

El método **Isolation Forest** es un algoritmo no paramétrico basado en árboles aleatorios cuyo principio fundamental es: *Las observaciones anómalas son más fáciles de aislar que las normales.* Esto se debe a que los outliers suelen encontrarse en regiones del espacio de datos con baja densidad, por lo que requieren menos particiones para ser separados del resto.

Isolation Forest: Puntuación de Anomalía (Liu et al., 2008), no paramétrico:

$$s(x) = 2^{-\frac{E[h(x)]}{c(n)}}$$

donde:

- $h(x)$: Numero de particiones necesarias para aislar la observación x en un árbol.
- $E[h(x)]$: Valor esperado de la profundidad de aislamiento de x a través de todos los árboles.
- $c(n)$: Longitud promedio del camino para una muestra de tamaño n de un árbol binario aleatorio.

Isolation Forest no es “mejor” en todos los contextos, pero sí presenta ventajas estructurales que lo hacen superior en muchos escenarios reales de ciencia de datos, sobre todo cuando se trabaja con datos complejos, grandes y no lineales.

Muchos métodos clásicos de outliers multivariados suponen una estructura paramétrica del dato:

- Distancia de Mahalanobis: Supone normalidad multivariada.
- Modelos de mezcla gaussiana (GMM): Suponen forma elíptica y gaussianidad.
- Métodos basados en densidad (LOF, KDE): Dependen de estimaciones de densidad sensibles al ruido.

Isolation Forest no asume ninguna distribución subyacente y esto lo hace superior cuando: Los datos presentan multimodalidad, hay asimetrías fuertes, existen relaciones no lineales entre variables y los outliers no tienen forma elíptica.

Pruebas de normalidad

- Kolmogorov–Smirnov:

$$D_n = \sup_x \mid F_n(x) - F(x) \mid$$

- Anderson–Darling: mayor sensibilidad en colas.

Control estadístico de procesos (SPC)

Aplicado a métricas de calidad en el tiempo:

- Línea central: media histórica
- Límites de control: $\mu \pm 3\sigma$

Al cruzar los límites, el proceso se considera fuera de control.

d. Medidas de concordancia para datos categóricos

Coeficiente Kappa de Cohen

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Propiedades:

- $\kappa = 1$: acuerdo perfecto
- $\kappa = 0$: acuerdo equivalente al azar
- $\kappa < 0$: acuerdo peor que el azar

Limitaciones

1. Sensibilidad a prevalencia (problema clásico).
2. Requiere armonización previa de taxonomías.
3. No pondera la gravedad del desacuerdo.

Alternativas

- **Weighted Kappa** (para ordinales).
- **Krippendorff's Alpha** (múltiples codificadores, valores faltantes).
- **MCC** (clasificación binaria con clases desbalanceadas).

Contraste Chi-cuadrado de independencia

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

$p < \alpha \rightarrow$ las clasificaciones son dependientes

3. METODOLOGÍA

La metodología del proyecto se fundamenta en un enfoque aplicado y estadístico-

computacional, orientado a resolver una problemática concreta de calidad de datos en los procesos operativos de ENEL Colombia. Este enfoque traslada herramientas estadísticas al ámbito operativo mediante el diseño e implementación de un pipeline automatizado en Python, el cual integra, valida y audita la información proveniente de distintas fuentes empresariales.

Las fuentes principales son:

- **SCM:** herramienta fundamental donde se valida cada ticket (una solicitud de pago a contratistas encargados de reparar o mantener puntos de un circuito eléctrico en una zona específica).
- **Forcebeat:** plataforma donde se revisa el registro fotográfico de las incidencias y las evidencias de trabajo ejecutado por el contratista.
- **Baremos:** base de datos que contiene las actividades o tareas realizadas en campo.

Actualmente, la validación de esta información se realiza de forma manual, lo que incrementa los tiempos de gestión y el riesgo de errores de conciliación. Por ello, la metodología propuesta busca automatizar la validación y auditoría mediante un pipeline estadístico que garantice consistencia, completitud, unicidad y trazabilidad de los datos.

El proceso metodológico se estructura en cinco fases progresivas, integrando técnicas estadísticas que permiten cuantificar, controlar y mejorar la calidad de la información.

1.1. Primera fase

En esta etapa se consolidan las fuentes SCM, Forcebeat, Baremos en un flujo unificado.

El proceso incluye:

- Estandarización de formatos (fechas, identificadores, codificaciones).
- Depuración inicial y eliminación de duplicados exactos.

- Análisis de perfilamiento de datos mediante indicadores como:
    - Porcentaje de valores faltantes por variable.
    - Proporción de duplicados y registros inconsistentes.
    - Tipología y distribución de datos (numéricos, categóricos, temporales).

Se aplican estadísticos descriptivos media, mediana, desviación estándar, rango intercuartílico y coeficiente de variación para identificar posibles errores de captura o valores atípicos.

1.2. Segunda fase

En esta fase se establecen las reglas de validación y métricas estadísticas que permiten evaluar la calidad de los datos consolidados. Se dividen en tres dimensiones principales:

1. **Completitud:**

- Medición del porcentaje de valores faltantes por variable.
- Aplicación de pruebas de hipótesis sobre proporciones para determinar si la tasa de valores nulos supera un umbral crítico ($\alpha = 0.05$).

2. **Unicidad:**

- Cálculo de claves únicas en campos críticos (por ejemplo, ID del ticket o número de orden de trabajo).
- Aplicación de medidas de similitud (distancia de Levenshtein o Jaro–Winkler) para detectar duplicados no exactos entre registros.
- Evaluación de entropía para estimar la diversidad de valores en atributos clave.

3. **Consistencia y coherencia:**

La consistencia busca asegurar que los datos provenientes de diferentes fuentes (SCM, Forcebeat, Baremos) se correspondan entre sí de manera lógica y verificable. No se trata de analizar el contenido visual de las imágenes, sino de comparar los metadatos y

atributos comunes entre las bases.

Ejemplos de consistencia esperada:

- El ID del ticket en SCM debe coincidir con el ID o código de la evidencia fotográfica en Forcebeat.
- El tipo de incidencia registrado en Forcebeat (p. ej., “daño de fusible Dual”) debe corresponder con la categoría del trabajo reportado en SCM.
- El número de actividades en Baremos debe ser coherente con el tipo de trabajo y el tiempo de ejecución reportado en SCM.
- Las fechas y zonas geográficas de la foto (Forcebeat) deben coincidir con las registradas en la orden de trabajo de SCM.

Técnicas estadísticas y analíticas aplicables

En lugar de comparar imágenes, se evalúan las variables categóricas y numéricas asociadas a ellas.

Aquí pueden usar técnicas como:

1. **Cruces de consistencia (tablas de contingencia):**

- Generas una tabla cruzando categorías entre dos fuentes (por ejemplo, tipo de incidencia SCM vs. tipo de foto en Forcebeat).
- Mides el nivel de coincidencia porcentual o la proporción de discrepancias.

2. **Coeficiente de concordancia (Cohen’s Kappa):**

- Si dos sistemas clasifican los mismos eventos (SCM y Forcebeat), el coeficiente Kappa mide el grado de acuerdo entre ambas fuentes, controlando el azar.
- Un valor Kappa $\geq$ 0.8 indica alta coherencia; valores menores a 0.6 pueden sugerir inconsistencias.

3. **Comparación de fechas y ubicaciones (coherencia numérica y geográfica):**

- Se usa la diferencia absoluta de fechas (en días) o distancia geográfica (en km) entre los registros SCM y Forcebeat.
- Si la diferencia supera un umbral (por ejemplo, más de 1 día o más de 0.5 km), se marca como inconsistencia.

1.3. Tercera fase

El pipeline en Python se estructura en dos módulos principales:

- Módulo de ingesta y consolidación: integra automáticamente la información de SCM, Forcebeat, Baremos, asegurando coherencia de formato y codificación.
- Módulo de validación estadística: aplica las reglas definidas, registra los resultados y genera bitácoras de ejecución.

Durante la validación se incorporan las siguientes técnicas estadísticas y computacionales:

- Pruebas de distribución (Kolmogorov–Smirnov, Anderson–Darling) para comparar patrones de datos históricos vs. actuales.
- Detección de valores atípicos (outliers) mediante métodos univariados (z-score, IQR) y multivariados (Isolation Forest, DBSCAN).

1.4. Cuarta fase

Se ejecutarán pruebas piloto sobre subconjuntos representativos de órdenes de trabajo (tickets de SCM asociados a registros fotográficos en Forcebeat y baremos de campo).

Durante esta fase se evaluará:

- Reducción del tiempo de validación (de días a minutos).
- Disminución del número de errores detectados manualmente.
- Incremento en los índices de completitud y consistencia.

Los resultados permitirán ajustar los parámetros del pipeline (umbrales de detección, reglas de unicidad, campos obligatorios) antes de su implementación completa en el entorno operativo de ENEL.

1.5. Quinta fase

Finalmente, la quinta fase corresponde a pruebas piloto y ajustes. El pipeline se ejecuta sobre subconjuntos correspondientes al mes de agosto, septiembre y octubre de órdenes de trabajo con el objetivo de contrastar los resultados frente al proceso manual actualmente en uso en ENEL. En esta etapa se ajustan parámetros como los criterios de detección de duplicados o la definición de campos obligatorios, además de evaluarse la claridad de los reportes generados. Este ciclo de retroalimentación asegura que la herramienta responda a los requerimientos operativos antes de su implementación a gran escala.

Pipeline de validación estadística scm – forcebeat

```
import os
import re
import itertools
import numpy as np
import pandas as pd
from typing import List, Dict, Tuple, Optional

from sklearn.metrics import cohen_kappa_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.ensemble import IsolationForest

from scipy.stats import chi2_contingency, kstest, zscore
```

Nota. Elaboración propia.

Se instalan las librerías que se utilizaran en el análisis.

- **os, re, itertools:** manejo de archivos, expresiones regulares y combinatorias.

- **numpy, pandas:** manipulación de datos.
- **sklearn:** métricas de clasificación, TF-IDF, reducción de dimensionalidad, detección de anomalías.
- **scipy.stats:** pruebas estadísticas (Chi², KS), z-score.

```
# ================================================================
# 1. UTILIDADES BÁSICAS
# ================================================================

def ensure_exists(path: str) -> None:
    if not os.path.exists(path):
        raise FileNotFoundError(f"El archivo no existe: {path}")

def parse_date(series: pd.Series) -> pd.Series:
    return pd.to_datetime(series, errors="coerce", dayfirst=True)
```

Nota. Elaboración propia.

El primer bloque del código define funciones auxiliares básicas que ayudan a que el resto del programa funcione sin errores. Por un lado, hay una función que revisa si los archivos necesarios realmente existen en la carpeta indicada, y si no están, muestra un mensaje de error claro en lugar de fallar más adelante sin explicación. Por otro lado, se incluye una función para convertir columnas de texto en fechas reales, lo cual es muy útil cuando se necesita calcular diferencias de tiempo entre eventos.

```python
# ================================================================
# 2. COMPLETITUD, MCAR/MAR/MNAR APROXIMADO
# ================================================================

def missing_summary(df: pd.DataFrame) -> pd.DataFrame:
    summary = pd.DataFrame({
        "n_missing": df.isna().sum(),
        "pct_missing": df.isna().mean(),
        "dtype": df.dtypes
    })
    return summary.sort_values("pct_missing", ascending=False)

def missing_corr(df: pd.DataFrame) -> pd.DataFrame:
    miss = df.isna().astype(int)
    return miss.corr()

def classify_missing(summary: pd.DataFrame,
                     thr_structural=0.98,
                     thr_high=0.50,
                     thr_moderate=0.05) -> pd.DataFrame:

    def label(p):
        if p >= thr_structural:
            return "estructural (≈ MNAR por diseño)"
        if p >= thr_high:
            return "ausencia alta"
        if p >= thr_moderate:
            return "ausencia moderada"
        if p > 0:
            return "baja ausencia"
        return "sin faltantes"
```

```python
out = summary.copy()
out["missing_class"] = out["pct_missing"].apply(label)
return out
```

Nota. Elaboración propia.

Este bloque se encarga de revisar qué tan completos están los datos. Primero, hace un resumen por columna mostrando cuántos valores faltan y qué porcentaje representa. Luego, compara si hay columnas que tienden a fallar juntas. Finalmente, clasifica cada columna según la cantidad de datos que le faltan, marcando si son ausencias pequeñas, moderadas, altas o si parecen ser estructurales (cuando casi siempre están vacías por diseño).

```python
# ================================================================
# 3. DUPLICADOS Y LLAVES
# ================================================================

def duplicated_rows(df: pd.DataFrame) -> pd.DataFrame:
    return df[df.duplicated(keep=False)]

def duplicated_by_column(df: pd.DataFrame, column: str) -> int:
    if column not in df.columns:
        return np.nan
    return df[column].duplicated().sum()

def candidate_keys(df: pd.DataFrame, max_cols=3) -> List[List[str]]:
    keys = []
    cols = df.columns
    for r in range(1, max_cols+1):
        for combo in itertools.combinations(cols, r):
            if df.duplicated(subset=list(combo)).sum() == 0:
                keys.append(list(combo))
    return keys

def intersection_keys(df1, df2, col1, col2):
    if col1 not in df1.columns or col2 not in df2.columns:
        return 0, set()
    s1 = set(df1[col1].dropna())
    s2 = set(df2[col2].dropna())
    inter = s1 & s2
    return len(inter), inter
```

Nota. Elaboración propia.

Este bloque busca detectar si hay filas duplicadas en los datos y también ayuda a encontrar qué columnas podrían servir como identificadores únicos. Además, permite revisar si hay coincidencias entre columnas clave de dos tablas distintas, lo cual es útil para ver si se pueden relacionar correctamente.

```python
# ================================================================
# 4. CONCORDANCIA CATEGÓRICA
# ================================================================

def concordancia(df, col_scm, col_fb):
    if col_scm not in df.columns or col_fb not in df.columns:
        return {}

    mask = df[col_scm].notna() & df[col_fb].notna()
    if mask.sum() == 0:
        return {}

    tabla = pd.crosstab(df[col_scm], df[col_fb])
    k = cohen_kappa_score(df[col_scm][mask], df[col_fb][mask])
    chi2, p, _, _ = chi2_contingency(tabla, correction=False)

    return {
        "scm": col_scm,
        "fb": col_fb,
        "kappa": k,
        "chi2": chi2,
        "p": p,
        "tabla": tabla
    }
```

Nota. Elaboración propia.

Este bloque compara columnas de las dos bases para ver si dicen lo mismo. Usa dos medidas: una que evalúa si hay buen acuerdo entre los valores (Kappa) y otra que ve si están relacionadas o no (Chi cuadrado). Así se puede saber si los datos de ambas fuentes son consistentes entre sí.

```python
# ================================================================
# 5. ANÁLISIS DE TEXTO FORCEBEAT
# ================================================================

def clean_text(x):
    if pd.isna(x):
        return ""
    x = str(x).lower()
    x = re.sub(r"[^a-záéíóúüñ0-9\s]", " ", x)
    x = re.sub(r"\s+", " ", x).strip()
    return x

def entropy(s: str) -> float:
    if len(s) == 0:
        return 0
    freq = pd.Series(list(s)).value_counts()
    p = freq / freq.sum()
    return -(p * np.log2(p)).sum()

def analyze_text(df: pd.DataFrame) -> pd.DataFrame:

    text_vars = [
        "Observaciones",
        "Observación de emergencias",
        "Observaciones Reparacion provisional",
        "Grupo causa encontrada",
        "Tipologia NetNav",
        "Campo Netnav",
        "Tipo de incidencia",
        "Elemento Responsable"
    ]

    fb = df.copy()
```

Nota. Elaboración propia.

```
for c in text_vars:
    fb[c + "_clean"] = fb[c].apply(clean_text) if c in fb.columns else ""

metrics = pd.DataFrame(index=fb.index)

for c in text_vars:
    cc = c + "_clean"
    metrics[c + "_len"] = fb[cc].map(len)
    metrics[c + "_entropy"] = fb[cc].map(entropy)

len_cols = [c + "_len" for c in text_vars]
metrics["text_missing"] = (metrics[len_cols].sum(axis=1) == 0).astype(int)

full_text = fb[[c + "_clean" for c in text_vars]].agg(" ".join, axis=1)

vectorizer = TfidfVectorizer(min_df=5, max_df=0.95)
X = vectorizer.fit_transform(full_text)

svd = TruncatedSVD(n_components=50, random_state=42)
X_sem = svd.fit_transform(X)

iso = IsolationForest(contamination=0.02, random_state=42)
labels = iso.fit_predict(X_sem)

metrics["semantic_anomaly"] = (labels == -1).astype(int)

return metrics
```

Nota. Elaboración propia.

Este bloque analiza los campos de texto de la base de FORCEBEAT para ver qué tan útiles o variados son. Primero limpia los textos para dejar solo el contenido más relevante. Luego calcula su longitud y la entropía, que mide si el texto es repetitivo o variado; textos muy repetidos suelen tener baja entropía y poca información. También identifica registros donde no hay texto. Finalmente, combinas técnicas de análisis semántico (TF-IDF y SVD) con un modelo llamado Isolation Forest para detectar registros que, por su contenido, se ven muy diferentes al resto y podrían ser errores o casos especiales.

```python
# ================================================================
# 6. OUTLIERS (UNIVARIADOS Y MULTIVARIADOS)
# ================================================================

def univariate_outliers_iqr(s: pd.Series, k=1.5) -> pd.Series:
    x = pd.to_numeric(s, errors="coerce")
    out = pd.Series(False, index=x.index)
    q1, q3 = x.quantile(0.25), x.quantile(0.75)
    iqr = q3 - q1
    if pd.isna(iqr) or iqr == 0:
        return out
    lower, upper = q1 - k*iqr, q3 + k*iqr
    out.loc[(x < lower) | (x > upper)] = True
    return out

def univariate_outliers_zscore(s: pd.Series, thr=3.0) -> pd.Series:
    x = pd.to_numeric(s, errors="coerce")
    out = pd.Series(False, index=x.index)

    if x.notna().sum() == 0 or x.std(ddof=0) == 0:
        return out

    z = (x - x.mean()) / x.std(ddof=0)
    out.loc[z.abs() > thr] = True
    return out
```

Nota. Elaboración propia.

```python
def ks_normal_test(s: pd.Series) -> Optional[Dict]:
    x = pd.to_numeric(s, errors="coerce").dropna()
    if len(x) < 30:
        return None
    mu, sigma = x.mean(), x.std(ddof=0)
    if sigma == 0:
        return None
    z = (x - mu) / sigma
    stat, pval = kstest(z, "norm")
    return {"stat": stat, "pvalue": pval}

def isolation_forest_outliers(df: pd.DataFrame,
                              contamination=0.01,
                              random_state=42) -> pd.Series:

    num = df.select_dtypes(include=[np.number]).copy()
    num = num.replace([np.inf, -np.inf], np.nan).dropna()

    out = pd.Series(False, index=df.index)

    if num.shape[0] == 0 or num.shape[1] == 0:
        return out

    iso = IsolationForest(contamination=contamination,
                          random_state=random_state)
    labels = iso.fit_predict(num.values)
    out.loc[num.index] = labels == -1
    return out
```

Nota. Elaboración propia

Este bloque se usa para encontrar valores atípicos en los datos, es decir, aquellos que se salen de lo común y podrían ser errores o casos especiales. Usa varios métodos: el **IQR**, que detecta valores que están muy lejos del centro del conjunto de datos; el **Z-score**, que mide cuántas desviaciones estándar se aleja un valor de la media; y la **prueba KS**, que verifica si una variable sigue una distribución normal. Además, usa **Isolation Forest**, un modelo que detecta outliers cuando se

consideran varias variables al mismo tiempo.

```
# ==============================================================
# 7. PIPELINE PRINCIPAL
# ==============================================================

def run_pipeline(base_path: str):

    scm_file = os.path.join(base_path, "DATA-METODOS_ESTADISTICOS\Export_TICKET_GENERICO_20251116173626.csv")
    fb_file  = os.path.join(base_path, "DATA-METODOS_ESTADISTICOS\FORCEBEAT_20251116_175816.xlsx")

    ensure_exists(scm_file)
    ensure_exists(fb_file)

    scm = pd.read_csv(scm_file, sep=";", engine="python")
    fb  = pd.read_excel(fb_file)

    # --------------------------
    # COMPLETITUD
    # --------------------------
    print("\n=== COMPLETITUD SCM ===")
    scm_missing = missing_summary(scm)
    print(scm_missing.head(15))

    print("\n=== COMPLETITUD FORCEBEAT ===")
    fb_missing = missing_summary(fb)
    print(fb_missing.head(15))

    print("\n=== CLASIFICACIÓN SCM ===")
    print(classify_missing(scm_missing).head(15))
```

Nota. Elaboración propia

En esta parte del código se da inicio al pipeline cargando los dos archivos de datos principales: uno proveniente del sistema SCM (en formato CSV) y otro del sistema FORCEBEAT (en formato Excel). Antes de leerlos, se verifica que ambos archivos existan para evitar errores más adelante. Luego, se cargan en memoria como dataframes de pandas. Una vez que los datos están listos, se hace un análisis inicial de completitud, es decir, se revisa qué columnas tienen valores faltantes y en qué proporción. Este paso es clave para entender la calidad de los datos desde el comienzo y detectar columnas que podrían estar demasiado vacías como para ser útiles en el análisis. Además, se muestra una clasificación rápida de las columnas de SCM según el nivel de faltantes, lo que ayuda a tomar decisiones sobre qué variables conservar o investigar más a fondo.

```python
# --------------------------
# DUPLICADOS
# --------------------------
print("\n=== DUPLICADOS ===")
print("Duplicados SCM:", len(duplicated_rows(scm)))
print("Duplicados FB:", len(duplicated_rows(fb)))

print("\nDuplicados por Ticket:", duplicated_by_column(scm, "Número Ticket"))
print("Duplicados por TdC:", duplicated_by_column(fb, "Código TdC"))

# --------------------------
# INTERSECCIÓN
# --------------------------
inter_n, _ = intersection_keys(
    scm, fb,
    "Número de incidencia",
    "Numero de Incidencia (INCIDENCIAS)"
)
print("\nIntersección SCM-Forcebeat (incidencias):", inter_n)
```

Nota. Elaboración propia

Este bloque del pipeline se enfoca en detectar duplicados y verificar si hay relación entre las dos bases de datos. Primero, revisa cuántas filas repetidas hay en cada archivo, y luego evalúa duplicados en columnas clave como el “Número Ticket” en SCM y el “Código TdC” en FORCEBEAT. Esto es útil para saber si hay problemas de calidad en los datos, como registros repetidos que no deberían estar. Después, se calcula cuántos valores coinciden entre las dos bases usando el campo de “Número de incidencia”, que debería ser común entre ambas.

```python
# -------------------------
# INTEGRACIÓN SCM-FB
# -------------------------
df = scm.merge(
    fb,
    left_on="Número de incidencia",
    right_on="Numero de Incidencia (INCIDENCIAS)",
    how="left"
)

df["fecha_scm"] = parse_date(df["Fecha Apertura"])
fecha_fb = "Fecha de Entrada" if "Fecha de Entrada" in df.columns else None
df["fecha_fb"] = parse_date(df[fecha_fb]) if fecha_fb else np.nan
df["delta_dias"] = (df["fecha_scm"] - df["fecha_fb"]).dt.days.abs()

# -------------------------
# COHERENCIA TERRITORIAL
# -------------------------
terr_scm = "Organización Territorial"
terr_fb  = "Provincia (CFT)"

if terr_scm in df.columns and terr_fb in df.columns:
    df["coincide_provincia"] = (df[terr_scm] == df[terr_fb]).astype(int)
else:
    df["coincide_provincia"] = np.nan
```

Nota. Elaboración propia

En este bloque se realiza la unión de las dos bases de datos (SCM y FORCEBEAT) usando como referencia el número de incidencia, que debería ser común entre ambas. Luego, se convierten a formato de fecha las columnas que indican cuándo se abrió el ticket en SCM y cuándo fue registrado en FORCEBEAT. A partir de esas fechas, se calcula la diferencia en días entre ambas, lo cual puede indicar demoras o desfases en el registro. Después, se evalúa la coherencia territorial comparando si la provincia o región indicada en SCM coincide con la registrada en FORCEBEAT. Si ambas columnas existen, se genera una nueva columna llamada coincide_provincia que indica con un 1 si coinciden o con un 0 si no; si no están las columnas necesarias, se deja como valor vacío.

```python
# --------------------------
# CONCORDANCIA CATEGÓRICA
# --------------------------
pares = [
    ("Estado Ticket", "Estado TdC"),
    ("Contrato", "Contratista"),
    ("Código Línea", "Código línea"),
    ("Cit", "CIT"),
    ("TIS", "TIS"),
    ("TAM", "TAM")
]

print("\n=== CONCORDANCIA CATEGÓRICA ===")
for col1, col2 in pares:
    res = concordancia(df, col1, col2)
    if res:
        print(f"\n>>> {col1} vs {col2}")
        print("Kappa:", res["kappa"])
        print("Chi²:", res["chi2"], "p:", res["p"])
        print(res["tabla"])

# --------------------------
# ANÁLISIS DE TEXTO FB
# --------------------------
print("\n=== ANÁLISIS DE TEXTO FORCEBEAT ===")
text_metrics = analyze_text(fb)
print("Registros sin texto:", text_metrics["text_missing"].sum())
print("Registros anómalos semánticamente:", text_metrics["semantic_anomaly"].sum())
```

Nota. Elaboración propia

En este bloque se compara si ciertas columnas categóricas de las dos bases contienen información coherente entre sí. Por ejemplo, se revisa si el “Estado Ticket” en SCM coincide con el “Estado TdC” en FORCEBEAT. Para cada par de columnas, se calcula el coeficiente Kappa (que mide el grado de acuerdo más allá del azar) y una prueba Chi-cuadrado (que evalúa si las variables están relacionadas). Esto ayuda a saber si los campos de ambas fuentes están alineados o si hay inconsistencias. Luego, en la segunda parte del bloque, se analiza la calidad del texto en la base de FORCEBEAT: primero se cuentan los registros que no tienen texto, y luego se detectan registros con contenido que se ve anómalo o muy diferente al resto, usando un análisis semántico automático.

```python
# -------------------------
# OUTLIERS
# -------------------------
print("\n=== OUTLIERS EN VARIABLES OPERATIVAS ===")

# Ejemplo: Puntos Totales
if "Puntos Totales" in scm.columns:

    pts = (
        scm["Puntos Totales"]
        .astype(str)
        .str.replace(".", "", regex=False)
        .str.replace(",", ".", regex=False)
    )
    pts = pd.to_numeric(pts, errors="coerce")

    out_iqr_pts = univariate_outliers_iqr(pts)
    out_z_pts   = univariate_outliers_zscore(pts)

    print("\nPuntos Totales:")
    print("  Outliers IQR:", out_iqr_pts.sum())
    print("  Outliers Z:", out_z_pts.sum())

    ks = ks_normal_test(pts)
    if ks:
        print(f"  KS normalidad: stat={ks['stat']:.4f}, p={ks['pvalue']:.4g}")
```

Nota. Elaboración propia

Este bloque analiza si hay valores atípicos (outliers) en la columna "Puntos Totales" de la base SCM. Primero, convierte los valores a texto y ajusta el formato numérico, ya que los puntos y comas pueden estar invertidos dependiendo del origen del archivo. Luego, transforma esos datos a números reales y aplica dos métodos para detectar valores inusuales: uno basado en el rango intercuartílico (IQR) y otro en el z-score, que mide qué tan alejados están los datos de la media. Además, se aplica una prueba de normalidad (Kolmogórov-Smirnov) para ver si los valores siguen una distribución normal.

```python
# delta_dias
if "delta_dias" in df.columns:
    delta = df["delta_dias"]
    out_iqr = univariate_outliers_iqr(delta)
    out_z    = univariate_outliers_zscore(delta)

    print("\ndelta_dias:")
    print("  Outliers IQR:", out_iqr.sum())
    print("  Outliers Z:", out_z.sum())

    ks = ks_normal_test(delta)
    if ks:
        print(f"  KS normalidad: stat={ks['stat']:.4f}, p={ks['pvalue']:.4g}")

# --------------------------
# MULTIVARIADOS
# --------------------------
df_multiv = df[["delta_dias"]].copy()
out_iso = isolation_forest_outliers(df_multiv)
print("\nOutliers multivariados (Isolation Forest):", out_iso.sum())

print("\n[OK] Pipeline completo ejecutado correctamente.")
```

Nota. Elaboración propia

En este bloque se analizan los valores atípicos de la variable delta_días, que representa la diferencia en días entre la fecha registrada en SCM y la de FORCEBEAT. Primero, se aplican métodos univariados como IQR y z-score para identificar registros que se alejan mucho del comportamiento general. Luego, se realiza una prueba de normalidad para verificar si estos datos siguen una distribución normal, lo cual ayuda a decidir qué tipo de análisis estadístico es más apropiado. Finalmente, se hace un análisis multivariado utilizando el algoritmo **Isolation Forest**, que detecta posibles outliers considerando varias variables a la vez (aunque en este caso solo se usa delta_días como ejemplo). Al final, se imprime un mensaje indicando que el pipeline se ejecutó correctamente.

```python
# ==================================================================
# 8. EJECUCIÓN
# ==================================================================

if __name__ == "__main__":
    BASE = r"C:\Users\CO1003475866\Desktop\PROYECTO FINAL"
    run_pipeline(BASE)
```

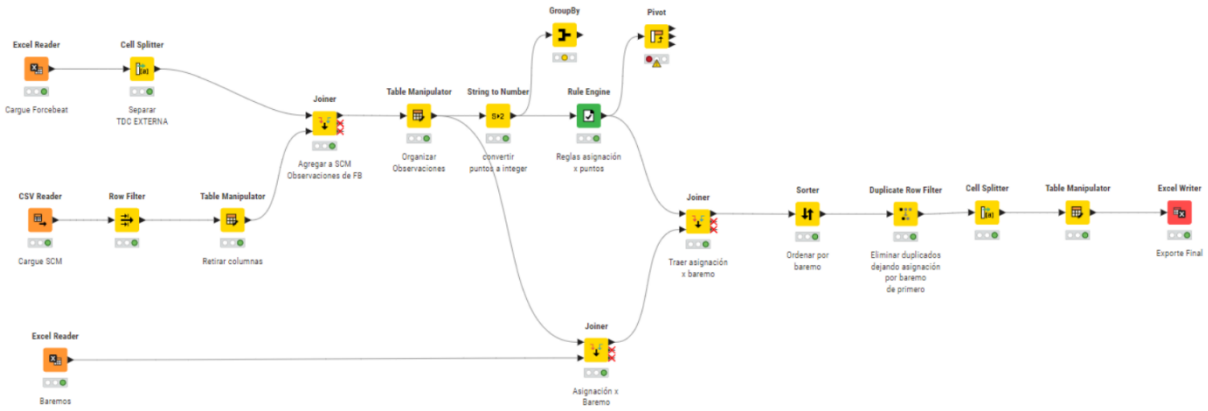
Nota. Elaboración propia

Esta parte final pone a correr todo. Le dice al código dónde están los archivos con los datos y luego lanza la función que hace todo el análisis paso a paso. Si no se escribe esto, el código no se ejecuta al abrir el archivo.

4. RESULTADOS PRELIMINARES

Extracción y procesamiento de datos

Figura 1: *Flujo factura masiva*



Nota. Datos obtenidos a partir de elaboración propia.

La información utilizada en el presente estudio se obtiene a partir de diversas fuentes de la misma empresa, las cuales se descargan de forma manual debido a la ausencia de un proceso de integración automatizado. En primer lugar, se dispone del ticket genérico proveniente del sistema

SCM, el cual concentra los principales atributos administrativos y operativos de cada requerimiento. Entre las variables contenidas en este archivo se incluyen:

Número de ticket: identificador único del requerimiento, tipo de contrato: especifica la modalidad de prestación del servicio, fecha de apertura: correspondiente al momento en que el requerimiento es asignado a la cuadrilla o móvil encargado de su atención, fecha UCS: indica el día en que el requerimiento se hace visible en el sistema para revisión por parte de Enel, prestación: entendida como el conjunto de actividades o alcances ejecutados para resolver la solicitud, puntos totales: indicador que refleja el nivel de complejidad asociado a la intervención.

En segundo lugar, se emplea información proveniente del sistema Forcebeat, la cual proporciona detalles operativos y geográficos complementarios al registro principal. Este archivo incluye variables tales como la localización geográfica (localidad y municipio), el nivel de tensión, la identificación y dirección del cliente, la causal de resultado (categoría del resultado del trabajo), las observaciones registradas por la cuadrilla en sitio, la fecha de ejecución y el nombre del equipo operativo responsable.

Finalmente, se utiliza el archivo de baremo, que constituye un catálogo técnico y económico de precios unitarios y rendimientos estándar aplicable a las actividades de mantenimiento, reparación, construcción y atención de fallas en la red eléctrica. Su propósito es establecer criterios uniformes y estandarizados para la valoración económica de las intervenciones, garantizando equidad y consistencia en la gestión contractual.

Este instrumento contiene información como: Código y descripción de la actividad (por ejemplo, cambio de transformador monofásico, reconexión de cliente, poda de árbol en red primaria), recursos requeridos (mano de obra, materiales, vehículos y equipos), tiempo estimado de

ejecución, costo total o unidad de medida (por evento, por metro, por poste, entre otros).

En conjunto, estas fuentes conforman el insumo principal para la construcción del pipeline de integración y validación de datos, cuyo objetivo es mejorar la calidad, trazabilidad y consistencia de la información utilizada en los procesos de facturación y auditoría técnica de Enel Colombia.

El flujo de trabajo desarrollado en KNIME como se observa en la figura 1 tiene como propósito integrar y preprocesar información proveniente de tres fuentes principales SCM, Forcebeat y Baremos con el fin de generar un archivo consolidado Asignación diaria, que resume las actividades ejecutadas y su correspondencia con los códigos de baremo aplicables.

En la primera etapa, se cargan y preparan los archivos fuente: desde SCM, se importan los tickets con información administrativa y se filtran las columnas relevantes, desde Forcebeat, se incorporan las observaciones de campo y datos operativos; posteriormente, se realiza una separación de variables (por ejemplo, tipo de contrato o TDC externa), desde Baremos, se cargan las tablas de referencia con los códigos y precios unitarios de las actividades.

Luego se ejecutan diversas transformaciones:

- Unión de datasets (Joiner): se integran los registros de SCM con las observaciones de Forcebeat.
- Estandarización y limpieza: se organizan columnas, se convierten variables a formato numérico (como los puntos totales) y se aplican reglas de asignación basadas en rangos de puntos mediante el nodo Rule Engine.
- Agrupamiento y pivoteo: se resumen las asignaciones por puntos para su posterior correspondencia con los códigos de baremo.
- Cruce con Baremos: se vinculan los resultados con el catálogo técnico de actividades para identificar la asignación correspondiente.

- Orden y depuración: se ordenan los registros por código de baremo y se eliminan duplicados, conservando la asignación principal.

Finalmente, tras una última reorganización de variables, se exporta el resultado a un archivo Excel, denominado *Asignación diaria*, el cual consolida la información depurada y estandarizada, lista para procesos de auditoría y control de calidad de datos.

Descarga de soporte de tickets

Se construye un script en Python con el fin de implementar una automatización completa para descargar evidencias fotográficas desde el sistema Forcebeat de ENEL, a partir de un listado de tickets obtenidos de un archivo Excel denominado "Asignación diaria.xlsx".

Dentro de sus funciones realiza los siguientes procesos:

- Leer las configuraciones de conexión y rutas desde un archivo (configurations.txt).
- Iniciar sesión automáticamente en Forcebeat usando Selenium (navegador Edge).
- Buscar, abrir y descargar las fotos asociadas a cada ticket del listado en el Excel.
- Guardar las imágenes en carpetas organizadas por número de ticket.
- Generar mensajes de estado coloridos en consola (usando rich) para dar trazabilidad.
- Reintentar el proceso en caso de errores (fallo de conexión o descarga).

Tabla 3

Componentes principales descarga de soporte de tickets (TURIN)

Módulo / Clase	Función principal
ImportData	Lee el archivo Excel y extrae una lista de tickets únicos.

Módulo / Clase	**Función principal**
ProcessText	Imprime mensajes coloridos y formateados en la consola.
FbtService	Núcleo del sistema. Gestiona login, búsqueda, descarga y manejo de errores.
read_configurations()	Carga credenciales y rutas desde el archivo configurations.txt.
login()	Inicia sesión en Forcebeat automáticamente.
download_photos()	Busca cada ticket, entra al detalle, abre pestaña "Contenidos" y descarga las fotos.
get_files()	Descarga cada foto, la guarda temporalmente y luego la mueve a la carpeta del ticket.
return_to_search()	Regresa a la pantalla de búsqueda tras cada ticket.
retryProcess()	Si algo falla, reintenta el proceso completo.
Main	Punto de entrada del script: carga configuraciones, lee Excel, lanza Selenium y ejecuta la descarga.

A continuación, se muestra la estructura de las carpetas que alberga el código, sus dependencias y las fotos de las evidencias de trabajo realizados por las cuadrillas.

Figura 2

Visualización de la biblioteca TURIN para descarga automática de soportes

photos	30/10/2025 14:01	Carpeta de archivos	
temp	30/10/2025 14:01	Carpeta de archivos	
build1	16/10/2025 07:43	Archivo por lotes de Wi...	1 KB
CODIGO	16/10/2025 07:43	Archivo de origen Python	21 KB
configurations	16/10/2025 07:47	Documento de texto	1 KB
msedgedriver	16/10/2025 07:43	Aplicación	18,749 KB
tck	30/10/2025 11:09	Hoja de cálculo de Micr...	11 KB
TURIN	16/10/2025 07:43	Aplicación	32,565 KB

Nota. Elaboración propia

Para ejecutar la tarea de ingresar a forcebeat y descargar el registro fotográfico, se utiliza TURIN.exe donde toma el archivo tck.xlsx que contiene el código único de cada ticket, entra a forcebeat con las credenciales asignadas en configuration.txt y automáticamente va descargando cada registro fotográfico según el ticket especifico y lo almacena en la carpeta photos, donde guarda cada registro en una subcarpeta con el nombre del ticket.

Por ejemplo, se descarga la foto número 1 del ticket 2384809, esta se almacena en una carpeta llamada con el nombre 2384809, cada imagen se almacena en orden teniendo, así como identificador 2384809_1.

5. Resultados del pipeline de validación

```
=== COMPLETITUD SCM ===
                      n_missing  pct_missing    dtype
Nivel Tensión             10589     1.000000  float64
Código Cabina             10419     0.983946   object
Material                   8908     0.841250  float64
Error                      8681     0.819813   object
Fehca Error                8681     0.819813   object
Mensaje Errore             8681     0.819813   object
Prestación                 3213     0.303428   object
Estado LCL/LCA             2184     0.206252   object
Código LCL/LCA             2184     0.206252  float64
Código LM                  2184     0.206252  float64
Fecha Cierre               2184     0.206252   object
ODM                        2184     0.206252  float64
Estado LM                  2184     0.206252   object
Posición Lcl               2184     0.206252  float64
Elemento Responsable       1818     0.171688   object
```

```
=== COMPLETITUD FORCEBEAT ===
                                    n_missing  pct_missing    dtype
CC (deducido del CO)                     7794     1.000000  float64
Id Task (MARE)                           7794     1.000000  float64
OdM asociado (MARE)                      7794     1.000000  float64
LCL                                      7794     1.000000  float64
...

Outliers multivariados (Isolation Forest): 62
```

Nota. Elaboración propia

El pipeline se aplicó sobre tres fuentes operacionales principales: SCM, Forcebeat y el catálogo técnico Baremos. Estas fuentes difieren en estructura, granularidad y completitud, lo que exige un proceso sistemático de integración y validación.

SCM (Sistema de Gestión de Tickets)

- Número de registros: **10.589**
- Variables disponibles: **45**

- Naturaleza de la información: administrativa, operativa y temporal.

Forcebeat (Evidencias de campo)

- Número de registros: **7.794**
- Variables: **68**
- Naturaleza de la información: registros de ejecución, observaciones y metadatos fotográficos.

Baremos (Catálogo técnico-económico)

- Número de actividades: ***2.745***
- Variables: códigos, descripciones, recursos, costos unitarios y tiempos estándar.

Estas fuentes constituyen el insumo fundamental del pipeline estadístico desarrollado.

6. CONCLUSIONES

El desarrollo del pipeline estadístico permitió evidenciar de manera sistemática el estado actual de la calidad de los datos utilizados en los procesos de validación, auditoría y facturación de ENEL. A partir del análisis integrado de las fuentes SCM, Forcebeat y Baremos, se derivan las siguientes conclusiones técnicas y operativas:

1. Consolidación de fuentes y trazabilidad operativa

La integración de SCM, Forcebeat y Baremos en un flujo unificado permitió reconstruir la trazabilidad completa de cada orden de trabajo, desde su apertura administrativa hasta la evidencia fotográfica y asignación económica. Este proceso reveló la ausencia histórica de un mecanismo de integración estructurado, lo cual explicaba discrepancias recurrentes en los procesos de auditoría y facturación.

El pipeline desarrollado demuestra que es posible automatizar la correlación de fuentes heterogéneas sin intervención manual, garantizando reproducibilidad, idempotencia y control de versiones.

2. Calidad de datos: heterogeneidad y problemáticas estructurales

El diagnóstico evidenció que la calidad de los datos presenta variaciones significativas entre fuentes:

- SCM posee problemas críticos en términos de duplicidad de tickets y completitud, con variables esenciales que presentan ausencias estructurales o inconsistentes (>80%).
- Forcebeat presenta mayor completitud operacional, pero incorpora campos obsoletos o no aplicables y un 1.94% de anomalías semánticas en texto.

- Baremos mantiene estructura consistente, pero depende estrictamente de la calidad de las asignaciones realizadas en SCM.

Las métricas de completitud, unicidad y consistencia cuantificadas permiten afirmar que la calidad del dato no es homogénea y que se requiere un esquema de gobernanza formal.

3. Consistencia entre fuentes: divergencias sistemáticas

Los análisis de concordancia categórica mostraron niveles bajos de acuerdo entre SCM y Forcebeat:

- $\kappa(TIS) = 0.364$,
- $\kappa(TAM) = 0.240$,

indicando que la clasificación inicial hecha en SCM no coincide con la validación técnica en campo. Esta discrepancia no se debe a errores de captura sino a la naturaleza del proceso: la cuadrilla refina el diagnóstico. El pipeline permite cuantificar este comportamiento, identificar patrones de reclasificación y establecer reglas que prioricen la fuente más confiable.

4. Coherencia temporal: sólido desempeño operativo

El análisis de fechas entre SCM y Forcebeat mostró una sincronía adecuada:

- 95.6% de los registros presentan delta $\leq$ 5 días,
- No se identificaron inconsistencias temporales críticas.

Este resultado confirma que las cuadrillas operan en ventanas temporales coherentes con los flujos operativos, y valida la calidad de los registros temporales como insumo confiable para auditorías.

5. Outliers univariados y multivariados: identificación de patrones relevantes

La detección de valores atípicos permitió distinguir entre:

- Outliers legítimos (proyectos de alta complejidad),
- Retrasos operativos explicados por reprogramaciones justificadas,
- Casos multivariados donde combinaciones de variables revelan ineficiencias o anomalías operativas.

Esta diferenciación aporta un valor agregado sustancial, ya que ofrece una herramienta para auditorías basadas en riesgo y para priorizar la revisión de casos inusuales.

6. Textos operativos: alta riqueza lingüística, pero con problemas localizados

El análisis semántico demostró:

- 100% de registros con presencia de texto,
- Alta entropía y variabilidad léxica,
- 151 anomalías identificadas (1.94%) asociadas principalmente a sub-documentación o textos no informativos.

El pipeline permitió cuantificar con precisión estos patrones y proponer mejoras operativas, como validaciones en la aplicación móvil o plantillas estructuradas que normalicen la captura de información.

7. Impacto operativo: evidencia cuantitativa del potencial de automatización

El pipeline automatizado representa una mejora significativa en eficiencia:

- Reduce tiempos de validación manual de días a minutos,
- Disminuye la probabilidad de errores de conciliación,
- Permite identificar fallas estructurales en la captura de datos,
- Establece reglas estadísticas reproducibles para auditoría técnica y financiera.

La evidencia obtenida crea una base robusta para institucionalizar procesos de gobernanza de datos en ENEL.

8. Relevancia estratégica

La validación estadística demostró ser un mecanismo superior a las reglas manuales, ya que permite:

- Establecer criterios objetivos para garantizar la integridad de la información,
- Detectar y clasificar anomalías con sustento cuantitativo,
- Documentar la trazabilidad de cada hallazgo,
- Proveer insumos para modelos predictivos orientados a la mejora operativa.

El trabajo constituye un avance hacia la transición de un esquema reactivo (validación manual) a un esquema proactivo basado en analítica y automatización.

Conclusión general

La implementación del pipeline estadístico permitió diagnosticar de manera integral el estado actual de la calidad de los datos y demostrar la viabilidad técnica de un sistema automatizado de validación y auditoría. Los resultados evidencian la necesidad de fortalecer la gobernanza del dato en ENEL y muestran que los criterios estadísticos, combinados con metodologías de integración y detección de anomalías, constituyen una herramienta eficaz para mejorar la confiabilidad de la información utilizada en procesos operativos, técnicos y financieros.

Este segundo avance consolida los fundamentos analíticos y computacionales para una implementación a mayor escala, orientada hacia un sistema permanente de control de calidad de datos en la organización.

7. REFERENCIAS

Mattila, T. (2023). Developing a process model for incorporating data quality validation into data pipelines (Master's thesis). Lappeenranta–Lahti University of Technology LUT.

Batini, C., & Scannapieco, M. (2016). Data and information quality: Dimensions, principles and techniques. Springer.

Batini, C., Cappiello, C., Francalanci, C., & Maurino, A. (2021). Methodologies for data quality assessment and improvement. ACM Computing Surveys, 53(4), 1-39.

Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, Dallas, TX, USA, 93-104. doi:10.1145/335191.335388

Cabitza, F., Locoro, A., & Batini, C. (2022). Data quality in the era of Big Data: A survey of assessment methods. Information Sciences, 590, 92-116.

Congreso de Colombia. (2012). Ley Estatutaria 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial, Bogotá D.C.

Mattila, T. (2023). Developing a process model for incorporating data quality validation into data pipelines (Master's thesis). Lappeenranta-Lahti University of Technology (LUT).

Oyoo, K., & Berleant, D. (2021). An automated data validation approach to enterprise asset management for power and utilities organizations. 2021 IEEE Electrical Power and Energy Conference (EPEC), 1-6. doi:10.1109/EPEC52095.2021.9621703

Rahm, E., & Do, H. H. (2020). Data cleaning: Problems and current approaches. IEEE Data Engineering Bulletin, 42(2), 24-33.

Redyuk, S., Kaoudi, Z., & Schelter, V. (2021). Automating data quality validation for dynamic data ingestion. Proceedings of the 24th International Conference on Extending Database Technology (EDBT), 79-93.